Feature: First deploy

From build-plan: feature 10b

Size: heavy - two separate risks with real consequences: putting the API, the dashboard and the database live with sign-in working across two addresses (cookies, the proxy's visitor address, a clean stop, the first login on an empty database), and then putting real data and a real Google calendar on it.

Branch: feature/10b-first-deploy

Status: verified 2026-10-23: both steps built, checked on the live app and reviewed; shared 196, backend 1052 and booking component 83 tests pass, the root build and the frontend build pass, the frontend lints.

Goal

Scheduleads runs for real use for the first time: the API and the database on Railway, the dashboard on Vercel, at the agency's own addresses. Frank signs in on the live app with an emailed code, AgentsWeb is set up there from its setup file, his real Google calendar is connected, and the booking package is published, so the agency site can install it the day its own plan allows. Deployed ahead of the agency site's launch, so backups (12c) and Primo (13) can follow (Frank, 2026-10-12).

In scope

- **Production readiness in the code** (only what a first deploy needs):
  - a one-time command that makes the platform admin login on a database with none (signup is closed and the seed refuses any live database, so today an empty live database has no way in);
  - the setup command (`client:setup`) opened to the live database behind an explicit flag, so it never touches it by accident;
  - the job runner's first start retried for a bounded time while Postgres is still coming up (F-179), instead of crash-looping until Railway stops restarting the API.
- **The deploy**: Railway's backend service and Postgres, Vercel's frontend, the two addresses under one domain with the session cookie shared between them, every production variable set, `NODE_ENV=production` , Railway's draining time set to 30 seconds (F-176), the migrations applied to the live database.
- **First-deploy checks, observed on the live app**: a clean stop at a redeploy (F-176), whether Railway's proxy replaces a visitor's own `X-Real-IP` (the rate limits depend on it), a restart with Postgres down (F-179), sign-in and sign-out across the two addresses.
- **AgentsWeb live**: the business made on `/admin/client-setup` , its setup file applied to the live database.

- **The one Google Cloud trip**: the live redirect, `http://localhost:3401/calendar/callback`, the `calendar.calendarlist.readonly` permission, then 12h's check against Frank's real calendar and the two findings carried for it (F-410, F-411).
- **The booking package published** to GitHub Packages (`@frankdmosquera/booking-component`, version from `package.json`), so a site can install it.

Out of scope

- Backups, restore and retention (12c, next, before any client's data).
- Google's app verification and leaving Testing mode (item 22's calendar half, before 13).
- The agency site's own launch: installing the package, its `WIDGET_ORIGINS` entry (its item 12).
- Texts on the live app: Twilio stays unset, so texts fail and wait, as the code already does, until a business needs them.
- Custom error pages, monitoring, alerts, analytics.

Build loop

Approved one step at a time (AGENTS.md). Heavy: `workflow.stepReview` is `every`, `checkpointCommits` enabled. After a step's plan has Frank's yes, nothing stops until the review, except the lines only he crosses, which in this feature are most of the work: every change on Railway, Vercel, the domain's DNS, Resend, Google Cloud and GitHub Packages, and every write to the live database, is done by Frank or on his yes at that moment, each one named in the step's plan beforehand so there is no surprise trip. The rest runs through: build, tests and checks, tick the box, write the build log entry and commit it to buildlogs, commit and push the step to `feature/10b-first-deploy`, `/audit` scoped to the step, then the independent review. The planned stop is after the review. Blocking findings (P0/P1) are fixed by default; P2/P3 are recorded and carried. `/complete` makes the final pass and merges on Frank's yes.

Build steps

- [x] **10b.1 Live, and Frank signs in.** "Frank opens the dashboard at its live address, asks for a code, gets it by email from the agency's address, and is in as the platform admin. He signs out and back in. The API's health answers at its own address. A redeploy while a job runs lets the job finish. A visitor who sends a made-up `X-Real-IP` is still limited as themselves."
  - Code: `packages/shared/scripts/create-platform-admin.ts` (`npm run admin:create --workspace=@scheduleads-app/shared -- <email>`): makes the one platform admin login, refuses when one already exists, prints the database it wrote to. `apply-client-setup.ts`: the live database only with `--live` (a database that is not local `*_dev` without `--live` is refused as today; with `--live` the dry run is still the default and `--apply` is still needed to write). `start-`

`job-runner.ts`: the first start retried for at most 60 seconds while Postgres refuses connections, then the API exits as today.
  - Frank's trips, in this order: the domain's DNS (two records), Railway (the backend service, its variables, the draining time, the domain), Vercel (the frontend, `NEXT_PUBLIC_API_URL`, the domain), Resend (the sending domain verified, if it is not already).
  - On his yes: `db:migrate` against the live database through the tunnel, then `admin:create` with his email.
  - Checks on the live app: `GET /health`; sign-in, sign-out, sign-in; a redeploy with a job in flight; the `X-Real-IP` check; a restart with Postgres stopped and started again (F-179). **Done when:** the new command's and the runner's tests pass, the setup command refuses a live database without `--live` (test), and every check above is observed on the live app and written in the log, with F-176 and F-179 closed or re-recorded with what was seen.
- ☑ **10b.2 AgentsWeb live, with Frank's real calendar.** "On the live app Frank makes AgentsWeb on `/admin/client-setup`, applies its setup file, connects his real Google calendar on Settings, Calendar and ticks the calendars that count. The live API's free times for AgentsWeb leave out his real appointments, and a test booking through the live API lands in his Google calendar and his inbox. The booking package is published."
  - Frank's trip: Google Cloud once (the live redirect `https://api.booking-crm.com/calendar/callback`, `http://localhost:3401/calendar/callback`, the `calendar.calendarlist.readonly` permission on the client's Data Access). Done 2026-10-21 (10b.2.1): the local redirect was already there; Data Access had no scopes at all, so all three the code asks for were added.
  - On his yes: `client:setup --live` dry run, then `--apply`, against the live database; the package published (`npm publish`, its version first).
  - Checks: 12h's pick against the real calendar (a second calendar ticked and unticked changes the free times); F-410 and F-411 looked at against the real account and closed or carried; a test booking made and then cancelled through its own link. **Done when:** every check above is observed on the live app and written in the log, and the package installs from GitHub Packages in an empty folder. Done 2026-10-22. The business is IA a la Orden (`ia-la-orden`, Frank bought ialaorden.com on Oct 21; AgentsWeb stays for the US and Canada later), owner login heguer76@gmail.com, renamed by hand and set up from `client-setups/ia-la-orden.ts`. Railway's security incident (Oct 21) came first: every secret was replaced and the API redeployed. Seen live: free times leave out Frank's appointments and follow the ticks; a test booking reached both inboxes and his calendar, then cancelled through its own link; booking emails from hola@ialaorden.com, notifications to his Gmail (ImprovMX refuses Resend's blacklisted IPs). F-176 closed on the evidence; F-410 and F-411 carried to the Calendar page's UI pass. Package 0.1.0 published and installed in an empty folder.

Files / areas

- `packages/shared/scripts/` (`create-platform-admin.ts` new, `apply-client-setup.ts`), its `package.json` scripts, `packages/shared/helpers/assert-local-dev-database.ts` (read, unchanged)
- `backend/lib/jobs/start-job-runner.ts` and its test
- `.env.example` (the production values named, never set), `AGENTS.md` Commands (the new command, the live flag, the live addresses)
- Outside the repo, Frank's: Railway, Vercel, the domain's DNS, Resend, Google Cloud, GitHub Packages

Data / contracts

- **`admin:create <email>`**: one `user` row with `role: "admin"`, `emailVerified: true`, name empty, as the seed makes it; refused with a message and exit code 1 when any user already has `role: "admin"`, or the email is not an email. Signs nobody in; the code comes at sign-in.
- **`client:setup <file> --live [--apply]`**: without `--live`, exactly today's guard; with it, any database, and the first line printed names the database's host and name before anything else is read.
- **The runner's first start**: retried while the database refuses connections, at most 60 seconds, then the error as today; any other error at once.
- **The live addresses**: the dashboard and the API on two subdomains of one domain; `COOKIE_DOMAIN` is that domain, `APP_ORIGIN` the dashboard, `BETTER_AUTH_URL` the API, `NEXT_PUBLIC_API_URL` the API. `WIDGET_ORIGINS` empty until a site goes live. The live `CALENDAR_TOKEN_KEY`, `BOOKING_LINK_KEY` and `BETTER_AUTH_SECRET` are new values, kept in the password manager, never the development ones.

Testing

`npm run test --workspace=@scheduleads-app/shared`, `npm run test --workspace=backend`, the frontend build and lint. One test per rule that would hurt if broken:

- 10b.1: `admin:create` makes one admin and refuses a second; the setup command refuses a database that is not local `*_dev` without `--live`, and with `--live` still writes nothing without `--apply`; the runner's first start retries while refused and gives up after its limit.
- 10b.2: no new code; its checks are observed on the live app.

Decisions (made while planning, Frank's to overrule)

1. **Deploy now, ahead of the agency site** (Frank, 2026-10-12): the plan's Oct 9 timing waited on QuickBooks, which nothing plans; 12c and 13 need a live app. 2. **The first login comes from a one-time command**, not from opening signup or a back door in the API: it runs from the laptop through the tunnel, like the migrations, and refuses once an admin exists. 3. **The setup command reaches the live database only with `--live`**: the guard that keeps it on `*_dev` stays the default, so a wrong `.env` line is still

refused. 4. **Railway's draining time 30 seconds** (F-176, Frank 2026-10-05), under which the API already stops itself at 25. 5. **The live domain is `booking-crm.com`** (Frank bought it 2026-10-12, in Vercel, so its DNS is there): the dashboard at `app.booking-crm.com` (Vercel), the API at `api.booking-crm.com` (Railway), the bare name kept free for a product page later. So `COOKIE_DOMAIN` is `.booking-crm.com`, `APP_ORIGIN` is `https://app.booking-crm.com`, and `BETTER_AUTH_URL` and `NEXT_PUBLIC_API_URL` are `https://api.booking-crm.com`. 6. **Google's consent screen stays in Testing for 10b** (Frank, 2026-10-21). Found while planning 10b.2: the Google Auth Platform app is in Testing, where Google drops a calendar connection after 7 days. Publishing needs a home page, a privacy policy link and `booking-crm.com` as an authorized domain, all empty, and the consent screen shows the app name `scheduleads-app` (the product name talk, parked). 10b.2's checks fit inside 7 days and AgentsWeb is the agency's own business, so it stays in Testing, and Frank reconnects when the dashboard shows the connection broke. Publishing (the privacy page, the name, Publish) is its own planned piece before a client connects a calendar: build-plan item 17, Face and Body.

Notes for the AI

- Railway builds from the repo root with `railway.json` (build, start, watch paths, health check, draining) and Node 26 from `.node-version` (2026-10-12: the dashboard's build command never reached the first build, and Railpack chose Node 24). Not the root `engines`: Vercel offers only Node 24 and 22, so the frontend runs on 24. Check its time zone data on the live `/b/` page in 10b.2 (Alberta on UTC-6 all year needs tz 2026c). Railway retires `railway.json` on 2026-12-01. Done 2026-10-23 another way: the converter would have made a second service ("scheduleads-app"), dropped the 30-second draining and needed a new package; the dashboard held every setting but the build command, and its build command never took (a deploy without `railway.json` ran only `npm install` and failed its health check, reverted in 9485cf9). So the build is a root `build` script, which Railway runs by itself, and `railway.json` is gone.
- The dashboard's watch paths skip commits that change only root files: such a commit deploys with `railway redeploy --service scheduleads --from-source`.
- Railway with Root Directory `backend` may not see `packages/shared`, which the backend's `prebuild` compiles. Check Railway's monorepo docs in 10b.1 before Frank's Railway trip; if it cannot, the service builds from the repo root with `npm run build --workspace=backend` and `npm run start --workspace=backend`, with watch paths on `backend/` **and** **`packages/shared/`**. The settled architecture (two deploy units, isolated) holds either way; say the mechanism in the step's plan. Vercel's `frontend` root must install the workspace from the repo root, since its `prebuild` needs `backend` and `packages/*`: check the same.
- Railway's proxy and `X-Real-IP`: `backend/lib/rate-limit/visitor-address.ts` trusts it in production. Checked by calling a public route past its limit with a different made-up `X-Real-IP` each time: limited means Railway replaces it.
- Production refuses to start without `RESEND_API_KEY` and `LOGIN_EMAIL_FROM`; the sending address must be on a domain verified in the agency's Resend account.

- Google OAuth stays in Testing: the connection needs reconnecting every seven days until item 22.

Open questions

None. The live addresses were answered by decision 5.

Implementation walkthrough

Feature 10b was specced heavy, two steps, and kept them. 10b.1's code is `bd239f2`. The deploy fixes it needed are `30fb983` to `9241467` (Railway settings and Node 26, the stale frontend lockfile, the Linux builds in the lockfile), then `6ed237b` (the node start) and `1e59286` (Better Auth's visitor header). Its live checks were ticked in `ce8474c`. Its review found F-430 to F-435; the fixes are in `b56dd77`, and the check of the fixes closed F-432 to F-435 in `aee9fa6`. 10b.2 added no code (`a9d6732`): the business went live as IA a la Orden, with Frank's real calendar. The Railway config ended in `5382dbe` (a root `build` script, `railway.json` removed), after a dashboard-only attempt that failed and was reverted (`34eccb3`, `9485cf9`). F-430 was marked fixed in `4ba03cd`, and the review checkpoint is `7654439`. The independent review of the whole feature passed, closed F-430 and raised one P3, F-436, which is carried.

the first login (`packages/shared/platform-admin/`, `scripts/create-platform-admin.ts`)

`createPlatformAdmin` works inside one transaction:

- It refuses when any user already has `role: "admin"`, when the email is already a login, or when it is not an email.
- Otherwise it inserts the admin the way the seed does: an empty name, `emailVerified: true`.

The script `admin:create <email>` prints `Database: host:port/name` before it reads anything. Its tests run inside rolled-back transactions (F-435), so they never leave an admin behind.

the setup command opened to the live database (`client-setup/setup-database.ts`)

`setupDatabase(url, { live })` keeps the `*_dev` guard unless `--live` is given. Even with `--live`, the command is a dry run until `--apply` is added. The command prints the database first. `databaseLabel` and `assertLocalDevDatabase` both parse through `parse-database-url.ts`, whose error never carries the URL, so a password cannot reach the terminal (F-434). The setup file `agentsweb.ts` became `ia-la-orden.ts`, the business that went live.

the runner's first start (`backend/lib/jobs/`)

`retryWhileDatabaseRefuses` retries `run()` every 2 seconds for at most 60 seconds, as long as Postgres refuses (`ECONNREFUSED`, `57P03`, or a Node `AggregateError` holding one of them). Any other

error is thrown at once (F-179). Whether the private host `postgres.railway.internal` fails with one of those codes was never seen live. That is F-431, still unverified.

the visitor's address (`backend/lib/rate-limit/`, `auth/auth-server.ts`)

Railway's proxy sets `X-Real-IP`, and the app's own limits already read it. Better Auth read `X-Forwarded-For`, found nothing there, and put every visitor in one bucket. It now reads the same header in production, through one constant, `VISITOR_ADDRESS_HEADER` (F-433), and a test pins that.

the deploy (`package.json`, `.node-version`, `AGENTS.md`, `.env.example`)

- **Railway** builds from the repo root. The root `build` script runs the backend build, and Railway runs it by itself. `.node-version` is 26.
- **What the dashboard holds:** the start command (`node backend/dist/server.js`, never through npm, F-176/F-432), draining 30 seconds with overlap 0, the health check and the watch paths. `railway.json` is gone, so each setting has one source (F-430).
- **Vercel** builds `frontend` on Node 24.
- `.env.example` names the live values: `api.` and `app.booking-crm.com`, with `COOKIE_DOMAIN` `.booking-crm.com`. It leaves out three the API needs to start (F-436).
- `AGENTS.md` gives the new commands, the live addresses and the Railway traps (a plain `railway redeploy` fails, and watch paths skip commits that touch only root files).

seen on the live app

- Sign-in, sign-out and sign-in again across the two addresses.
- `GET /health` answers.
- A made-up `X-Real-IP` is replaced by Railway, so the visitor is still limited as themselves.
- A restart with Postgres down came back.
- The stop signal reaches the API (`[api] SIGTERM: stopping`).
- IA a la Orden's free times leave out Frank's real appointments and follow the ticked calendars.
- A test booking reached both inboxes and his calendar, and was cancelled through its own link.
- `@frankdmosquera/booking-component` 0.1.0 is published and installs in an empty folder.

Railway's security incident (Oct 21-22) came first: every secret was replaced and the API was redeployed.

carried

These stay in the ledger:

- F-179 stays fixed, waiting on F-431.
- F-410 and F-411 go to the Calendar page's UI pass.
- F-436 is the `.env.example` gap.

- Google's consent screen stays in Testing until Face and Body (decision 6).

Findings

10b/F-176 [P2] closed - A deploy's clean stop depends on Railway's grace period, which nothing has confirmed

File: backend/server.ts:39-49 (spec: decision 9, "a job left mid-run by a crash (not a deploy, which stops cleanly)") **Found:** 2026-10-05 by independent review of step 8a.1 (scope: 779512a..17a9118; lenses: quality, security, performance, tests) **Why it matters:** On SIGTERM the API waits for `runner.stop()` with no time limit, then exits. Railway sends SIGKILL after `RAILWAY_DEPLOYMENT_DRAINING_SECONDS`; its docs page on teardown names the variable but this review could not confirm its default. If the window is shorter than a running job (a Google call or an email send), the job is killed while locked and is only taken again when graphile-worker treats the lock as abandoned (about four hours), so a confirmation could arrive hours late, the case decision 9 says a deploy avoids. The spec's Notes already say the SIGTERM path is first seen on Railway. Missing validation: Railway's default draining time and one observed deploy with a job in flight. **Suggested fix:** Confirm the default and set `RAILWAY_DEPLOYMENT_DRAINING_SECONDS` (for example 30) when the runner first runs on Railway; optionally bound the wait in `stop()` so the API exits itself before the SIGKILL. **Resolution:** Confirmed 2026-10-05 from Railway's documentation (deployment teardown): an old deployment gets SIGTERM, then SIGKILL after RAILWAY_DEPLOYMENT_DRAINING_SECONDS, about 0 to 3 seconds by default. A job in flight at a deploy can be killed and then waits about 4 hours for its lock to expire. The fix is a Railway setting on the backend service (for example 30 seconds), which only Frank changes; raised with him after 8a.1's review. Frank, 2026-10-05: set at the deploy that ships 8a; written into the spec's deploy notes. Stays open until then. Fixed 2026-10-21, observed on the live API (10b.1). Three stops (2026-10-19 and 2026-10-20) showed no `[api] SIGTERM: stopping`: Railway ran `npm run start`, npm passed the signal to `sh -c node`, and the shell died without passing it on, so the API was cut off. Railway's dashboard settings overrode `railway.json`'s deploy section (the deployment's settings read start command `npm run start --workspace=backend`, draining unset). `railway.json` now starts `node backend/dist/server.js` (6ed237b), and Frank set the same start command and Teardown (overlap 0, draining 30) in the dashboard. A `railway restart` at 01:23 UTC then logged `[api] SIGTERM: stopping` and came back healthy. No job was in flight (the live database has no business until 10b.2); the stop path itself is what was unproved. Independent review of 10b.1, 2026-10-21: left `fixed`, not closed. The draining time and the signal reaching the API are seen, but the case this finding is about, a job in flight at a deploy, was not (the step's own check "a redeploy while a job runs lets the job finish"), and the rule that makes the repair hold (start with `node`, never through npm) is written nowhere a later session reads (F-432). Close once a redeploy with a job in flight is seen in 10b.2 and F-432 is done. Closed by Frank, 2026-10-22 (10b.2.4), on the evidence rather than a timed redeploy: a booking email leaves within about a second of the booking, so catching one in flight at the instant a deploy swaps is luck. Seen: the stop signal reaches the API and it stops cleanly (Oct 21); the redeploy of Oct 22 (secret swap) came back

healthy with the runner working; and a job cut off anyway stays in the database queue and runs again once its lock expires (delayed, never lost). F-432 was fixed in b56dd77.

10b/F-430 [P3] closed - railway.json says how the API starts and drains, but Railway runs the dashboard's values

File: railway.json:15-21 **Found:** 2026-10-21 by /audit (scope: current, c89e355..ce8474c; lenses: quality, security, performance, tests) **Why it matters:** On this service the dashboard's settings beat the file's deploy section: the deployments built after 6ed237b still ran `npm run start` with draining unset until Frank typed the same values into the dashboard (each deployment's `serviceManifest`). The watch paths behave the same way. So the file now restates values that do not take effect from it, and the next edit to `railway.json` (or to `.railway/railway.ts`, due before Dec 1) will look applied while the dashboard keeps the old value. It bit three times in 10b.1 (the missing build command, the skipped root-only commit, the start command and draining time). **Suggested fix:** At the `.railway/railway.ts` migration planned in 10b, find which source the service honours (Railway's config-as-code setting for this service, or a `serviceManifest` read after a change to the file alone) and keep that one source; until then, a line in AGENTS.md's live-app paragraph that the dashboard wins and that `railway deployment list --json` (`serviceManifest`) is the check. **Resolution:** Fixed 2026-10-23 (10b): one source per setting. `railway.json` is deleted; start, draining, health check and watch paths live in the dashboard, and the build is the root `build` script, which Railway runs by itself (5382dbe, a deploy showed install then build and passed its health check). The dashboard build command never took: a deploy without the file ran only `npm install` and failed, reverted in 9485cf9. Closed 2026-10-23 by the independent review of feature 10b (c89e355..7654439): `railway.json` is absent at the target and `git grep` finds no other Railway config file, so nothing in the repo restates a dashboard setting; the one Railway value the repo holds is the root `build` script (package.json), which is the build's only source, and AGENTS.md says the dashboard holds start, draining, health check and watch paths and that `serviceManifest` is the check. `npm run build --workspace=backend` passes; the repair adds no new defect.

10b/F-432 [P2] closed - The rule that keeps the clean stop working, start the API with node and never through npm, is written nowhere a later session reads

File: AGENTS.md:477; railway.json:16; backend/server.ts:43-45 **Found:** 2026-10-21 by independent review of step 10b.1 (scope: c89e355..ebf0b33; lenses: quality, security, performance, tests) **Why it matters:** F-176's repair rests on one fact: through `npm run start` the SIGTERM never reaches the API (npm hands it to `sh -c`, which dies without passing it on), so a deploy cuts jobs off. That fact lives only in findings.md and the message of 6ed237b. AGENTS.md's Commands still give `npm run start --workspace=backend` as the backend start, railway.json cannot hold a comment, and server.ts's stop comment says nothing of it. The next tidy of railway.json or of the Railway dashboard to the documented command brings F-176 back with nothing failing, only a missing `[api] SIGTERM: stopping` line. **Suggested fix:** One line in AGENTS.md beside the backend start (and in the live-app paragraph): on Railway the API starts as `node backend/dist/server.js`, never through npm, because npm does not pass SIGTERM on; the same in the server.ts comment above `STOP_LIMIT_MS`. **Resolution:** Fixed 2026-

10-21: AGENTS.md now says, beside the live app, that Railway starts the API with `node backend/dist/server.js` and never through npm (F-176), that the dashboard wins over `railway.json` and `serviceManifest` is the check (F-430), and that a plain redeploy fails; the local backend start line says Railway starts it differently. Closed 2026-10-21 by the check of the fixes (ebf0b33..b56dd77): AGENTS.md, which every session loads, now states the node start, why npm breaks the stop signal, and that the dashboard wins, and railway.json already starts with `node backend/dist/server.js` ; the server.ts comment was not touched, which leaves no gap since AGENTS.md carries the rule.

10b/F-433 [P3] closed - The trusted visitor header is named in two places, and Better Auth's side has no test

File: backend/lib/auth/auth-server.ts:208; backend/lib/rate-limit/visitor-address.ts:9-12 **Found:** 2026-10-21 by independent review of step 10b.1 (scope: c89e355..ebf0b33; lenses: quality, security, performance, tests) **Why it matters:** Which header names the visitor, and that it is trusted only in production, is one decision now written twice: `visitorAddress` for the app's own limits and `advanced.ipAddress.ipAddressHeaders` for Better Auth's sign-in limits. A change of proxy (another host, a CDN in front) that updates one leaves the other on a header the new proxy does not set, or one a visitor can send; Better Auth then falls back silently to one shared bucket for everyone (its rate limiter warns once, @better-auth/core dist/utils/ip.mjs `getIP` returns null). No test pins the Better Auth setting, which is why its absence was only found on the live app. **Suggested fix:** One exported constant (for example the header name beside `visitorAddress`) used by both, so the two can only change together. **Resolution:** Fixed 2026-10-21: one name, `VISITOR_ADDRESS_HEADER` in backend/lib/rate-limit/visitor-address-header.ts, read by visitor-address.ts and auth-server.ts; backend/lib/auth/auth-visitor-address.test.ts checks Better Auth uses it in production and its own default elsewhere (proved able to fail with `x-forwarded-for` in its place). Closed 2026-10-21 by the check of the fixes (ebf0b33..b56dd77): `git grep` finds the header literal only in visitor-address-header.ts, both readers import it, the new test passes in production and development, and public-rate-limit-middleware.test.ts still pins the value Railway sends.

10b/F-434 [P3] closed - databaseLabel copies the label assertLocalDevDatabase builds, and a DATABASE_URL that does not parse prints its password

File: packages/shared/helpers/database-label.ts:3-5; packages/shared/helpers/assert-local-dev-database.ts:14-20; packages/shared/scripts/create-platform-admin.ts:23 **Found:** 2026-10-21 by independent review of step 10b.1 (scope: c89e355..ebf0b33; lenses: quality, security, performance, tests) **Why it matters:** `assertLocalDevDatabase` still builds `host:port/name` inline for its refusal while the new `databaseLabel` builds the same string, so the two can drift on how a database is named. Both call `new URL(url)` unguarded, and Node's `ERR_INVALID_URL` carries the whole URL in its `input` property (checked locally: a password containing `#` makes `new URL` throw with `input` set to the full URL, password included). `admin:create` calls it at the top level, so the uncaught error prints the password to the terminal. Reachable only with a password holding URL-special characters, which Railway's generated ones do not. **Suggested fix:** Have `assertLocalDevDatabase` use `databaseLabel`

for its message, and have `databaseLabel` catch the parse error and throw its own "DATABASE_URL is not a valid URL" without the input. **Resolution:** Fixed 2026-10-21: packages/shared/helpers/parse-database-url.ts replaces Node's URL error with one that carries no input; databaseLabel and assertLocalDevDatabase both use it, and the refusal message uses databaseLabel instead of its own copy. database-label.test.ts checks that a password with `#` is not in the error (proved able to fail on the old code). Closed 2026-10-21 by the check of the fixes (ebf0b33..b56dd77): no other unguarded `new URL` of a database URL remains outside tests, Node was confirmed to put the full URL in its error, and the replacement error carries neither input nor cause.

10b/F-435 [P3] closed - One admin:create test runs outside its undo, so on a database without the seed's admin it leaves a real admin login behind

File: packages/shared/platform-admin/create-platform-admin.test.ts:52-55 **Found:** 2026-10-21 by independent review of step 10b.1 (scope: c89e355..ebf0b33; lenses: quality, security, performance, tests) **Why it matters:** "refuses on the seeded database" calls `createPlatformAdmin(db, ...)` on the real connection, not inside `onDatabaseWithNoAdmin`'s rolled-back transaction. On a freshly migrated `*_dev` database before `db:seed`, or one whose seed admin was changed, the call succeeds, the test fails, and `another-admin@example.com` stays as the platform admin, so later `admin:create` runs and seed-based checks behave differently. It also only repeats the refusal the first test already proves. **Suggested fix:** Drop it, or run it inside a transaction that is rolled back, as the other two are. **Resolution:** Fixed 2026-10-21: the test runs inside a rolled-back transaction like the others (`rolledBack`), so it writes nothing on any database. Closed 2026-10-21 by the check of the fixes (ebf0b33..b56dd77): the call now goes through `tx` inside `rolledBack`, which always throws Undo, and all 196 shared tests pass.

Independent review

One receipt, over the whole feature at the spec-verified checkpoint. The steps' own reviews (10b.1's independent review and the check of its fixes) are recorded in the findings above and in the build log.

Status: passed **Target commit:** 76544392e872b373b362458b71e83c30a7f88e0d **Base commit:** c89e355ecb85736a7d1c0e02e15a4ebdc9503735 **Base ref:** main **Spec hash:** fecaa7d723e532b191885acc77647f5fc24a8cdb5fc45ab7a3e8b58003a0f7e3 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-23T01:00:01Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-23T01:03:54Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete `c89e355ecb85736a7d1c0e02e15a4ebdc9503735..76544392e872b373b362458b71e83c30a7f88e0d` delta

in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `npm run test --workspace=@scheduleads-app/shared` : pass (29 files, 196 tests)
- `npm run test --workspace=backend` : pass (91 files, 1052 tests, against the local seeded scheduleads_dev)
- `npm run test --workspace=@frankdmosquera/booking-component` : pass (16 files, 83 tests)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass
- `npm run lint --workspace=frontend` : pass

Evidence

- Freshness: HEAD = target; `git merge-base main HEAD` = base; current-feature.md SHA-256 matches; only `blueprint/context/review.md` differed from the target before this pass.
- Whole delta read (34 files, 22 commits): the admin:create command and its rolled-back tests, client:setup's `--live` guard (`setupDatabase`), the runner's bounded first-start retry, Better Auth's visitor header (`VISITOR_ADDRESS_HEADER` , production only), the password-free database label, the root `build` script, `.node-version` , `.env.example` , AGENTS.md, the plan docs and the lockfile change.
- Lockfile: only one resolved version moved (hoisted `typescript` 7.0.2 to 6.0.3); every workspace still resolves its declared TypeScript (backend, shared, booking-component 7.0.2 nested; frontend 5.9.3). The rest is the removed frontend lockfile folded into the root one plus added platform binaries.
- No `.only` , `.skip` or `.todo` added in the delta; the trusted header literal exists only in `visitor-address-header.ts` ; no unguarded `new URL` of a database URL remains outside tests; no leftover `railway.json` reference outside docs that say it is gone.
- Better Auth 1.7.5 `getIP` validates the header value as an IP; graphile-worker 0.18.0 `RETRYABLE_ERROR_CODES` re-read for F-179 and F-431.
- No secret was printed: `.env.example` was read by variable name only; the root `.env` was not read.

Findings

- F-436 [P3] open: the live variables in `.env.example` leave out RESEND_API_KEY, GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET, which the live API needs to start
- F-430 [P3] fixed -> closed (railway.json gone, one source per Railway setting)
- F-179 [P3] re-examined, left fixed (decided by F-431, still unverified)
- No P0 or P1 open or fixed

Remaining risk

- Check was not required and was not run; the live-app behaviour (sign-in across the two addresses, the X-Real-IP replacement, the clean stop, the real calendar) rests on the observations recorded in the build log, not re-observed here.
- F-431 (unverified): the error the live private host `postgres.railway.internal` gives while Postgres is down is still unknown, so the runner's 60-second wait may not cover it.
- The root `build` -script mechanism and the dashboard's start, draining and watch-path values live on Railway and were not inspected (no Railway access in this review).
- No browser tests, CI or Verify command exist in this project; the frontend has no test script.
- `blueprint/.state/run.json` was not written by this reviewer (writes limited to findings.md and review.md).